USRP X310 + TwinRX Configuration on Ubuntu and Python

Rubem Vasconcelos Pacelli
Aeronautics Institute of Technology

October 19, 2026

1 Hardware used

The setup considered here is:

- Ettus USRP X310;
- one TwinRX Rev. C receive daughterboard;
- TwinRX provides two receive front-ends/channels: RX0 and RX1;
- the second X310 daughterboard slot is empty;
- Ubuntu host computer connected to X310 port 0 through Ethernet;
- external 10 MHz frequency reference connected to `REF IN`;
- no external PPS is connected;
- Python 3.12 with UHD's Python bindings (`python3-uhd`) and the `sigmf` package, managed with `uv`.

The TwinRX is receive-only. Therefore, the current setup has two usable receive channels and no usable RF transmit chain.

2 Meaning of the main network addresses

The Ubuntu host and the X310 must have different IP addresses on the same subnet:

Device	Address	Purpose
Ubuntu Ethernet interface	`192.168.10.1/24`	Host-side address
USRP X310 port 0	`192.168.10.2`	Radio address
Subnet mask	`255.255.255.0`	Equivalent to `/24`

3 Create a persistent Ubuntu network profile

In Ubuntu, open:

Settings → Network → Wired → gear icon

Create or edit a profile with the following parameters:

Parameter	Value
Profile name	`usrp_x310_network_profile`
IPv4 method	Manual
IPv4 address	`192.168.10.1`
Netmask	`255.255.255.0`
Gateway	leave blank
DNS	leave blank
MTU	`1500`
Connect automatically	enabled

4 Useful Linux network checks

Check the interface state:

```
ip -br addr show enp109s0
```

Replace `enp109s0` with the actual NIC (Network Interface Controller/Card) name if it is different in your machine. A correct configuration should show approximately:

```
enp109s0   UP   192.168.10.1/24
```

Check the physical Ethernet carrier:

```
cat /sys/class/net/enp109s0/carrier
```

Interpretation:

- `1`: physical Ethernet link detected;
- `0`: no physical Ethernet carrier.

The same check can be performed with:

```
ethtool enp109s0 | grep "Link detected"
```

For a working 1 GbE link, `ethtool enp109s0` should normally report:

```
Speed: 1000Mb/s
Duplex: Full
Link detected: yes
```

Check routing to the radio:

```
ip route get 192.168.10.2
```

The route should use `enp109s0`, not Wi-Fi. A correct result is approximately:

```
192.168.10.2 dev enp109s0 src 192.168.10.1
```

Test connectivity:

```
ping -c 4 192.168.10.2
```

5 Verify the X310 with UHD

Install UHD host utilities if necessary:

```
sudo apt update
sudo apt install uhd-host python3-uhd
```

Discover the radio:

```
uhd_find_devices --args="addr=192.168.10.2"
```

For the present radio, the relevant output is:

```
product: X310
type: x300
addr: 192.168.10.2
fpga: HG
```

The value `type: x300` is the UHD family identifier used for both X300 and X310 devices.

Probe the complete hardware configuration:

```
uhd_usrp_probe --args="addr=192.168.10.2"
```

The expected TwinRX section contains:

```
RX Dboard: TwinRX Rev C
RX Frontend: 0  -> TwinRX RX0
RX Frontend: 1  -> TwinRX RX1
Sensors: lo_locked
```

Thus, one physical TwinRX daughterboard contains two receive front-ends/channels.

6 Python environment setup

Recording is done with a small set of Python scripts (`x310usrp_record.py`, `check_levels.py`, `report_summary.py`) in the project repository, managed with `uv`. UHD's Python bindings come from the `python3-uhd` apt package and only work with the system Python 3.12, so the `uv` virtual environment must be created with system site-packages enabled rather than a fully isolated one:

```
cd usrp-x310
rm -rf .venv
uv venv --python /usr/bin/python3 --system-site-packages
uv sync
```

`uv sync` installs the project's remaining dependencies declared in `pyproject.toml`: `numpy` (version-pinned to stay ABI-compatible with the system numpy that `python3-uhd` was built against) and `sigmf`, the official SigMF-project library used to build and validate the recording files.

7 Recording I/Q samples with `x310usrp_record.py`

`x310usrp_record.py` streams both TwinRX channels straight to disk and bundles the result into a single SigMF Archive. Typical usage:

```
uv run x310usrp_record.py --duration 5                          # L1 + L5, 5 s
uv run x310usrp_record.py --freq 1575.42e6 --channels 0         # L1 only
uv run x310usrp_record.py --iq-rate 4e6 20e6 --gain 25 30 --name test1  # per channel
uv run x310usrp_record.py --help                                 # all options, with
    defaults
```

The defaults reproduce the dual-frequency GNSS setup used in this lab: GPS L1 (1575.42 MHz) on RX0 at 4 MS/s and GPS L5 (1176.45 MHz) on RX1 at 20 MS/s, with the external 10 MHz clock reference and 25 dB of RX gain on both channels (see Section 8). `-freq`, `-gain` and `-iq-rate` each take either one value (applied to every channel) or one value per channel; when `-iq-rate` is omitted, each channel's rate is chosen from the band it is tuned to (4 MS/s for L1, whose C/A main lobe is about 2 MHz wide, and 20 MS/s for L5, whose 10.23 Mchip/s main lobe is about 20 MHz wide).

Because the two channels run at different sample rates, each channel is received through its own UHD streamer, in its own thread, and written to its own `.sigmf-data` file. Both streamers are

started by the same timed stream command, so the two recordings still begin at the same instant on the USRP's clock. Every per-channel quantity – sample rate, gain, frequency, written and lost samples – is stored in that channel's own `.sigmf-meta` and reported per channel.

7.1 SigMF Archive output

Each run produces a single file, `recordings/<name>.sigmf`, following the SigMF specification's Archive → Collection → Recording structure: the `.sigmf` file is a plain tar archive containing one `.sigmf-collection` JSON file (linking the two channels' recordings by name and SHA-512 hash) and, for each channel, one `.sigmf-meta` (JSON metadata: that channel's sample rate, frequency, gain, antenna, motherboard/daughterboard, start time, lost/written sample counts) and one `.sigmf-data` file (interleaved `int8` I/Q, SigMF datatype `ci8`: 1 byte for I and 1 byte for Q, 2 bytes per complex sample). Listing one archive's contents:

```
tar -tvf recordings/x310_20261009T160000Z.sigmf
```

```
x310_20261009T160000Z/x310_20261009T160000Z.sigmf-collection
x310_20261009T160000Z/x310_20261009T160000Z_ch0.sigmf-meta
x310_20261009T160000Z/x310_20261009T160000Z_ch1.sigmf-meta
x310_20261009T160000Z/x310_20261009T160000Z_ch0.sigmf-data
x310_20261009T160000Z/x310_20261009T160000Z_ch1.sigmf-data
```

Samples are carried as `sc8` both over the Ethernet link and on disk: the X310's FPGA converts the radio's 16-bit samples to 8 bits before sending them, so the default 4 + 20 MS/s costs 2 bytes × 24 MS/s = 48 MB/s, about half of the roughly 100 MB/s a 1 GbE link can carry (with 16-bit samples the same configuration would sit right at that limit). If the host falls behind anyway (an overflow), the lost samples are replaced by zeros, so a sample's position in the file always matches its time and the two channels stay aligned; each gap is logged on the console and recorded in that channel's `.sigmf-meta` as an annotation covering the zero-filled samples, and in its `x310:lost_samples` total. Recordings made before 19/10/2026 skipped lost samples instead of zero-filling them, and over-counted `x310:lost_samples`. X310 rates are 200 MHz divided by an integer (4 and 20 MS/s are exact: 200/50 and 200/10), so other requested rates may be coerced; the script prints the actual value used per channel.

At these rates a 300 s recording takes about 2.4 GB for L1 and 12 GB for L5, i.e. about 14.4 GB per recording and 29 GB per day.

7.2 Loading the recorded samples in Python

Each channel's complex I/Q samples can be read directly out of the archive without extracting it to disk first:

```
import tarfile
import numpy as np

with tarfile.open("recordings/x310_20261009T160000Z.sigmf") as tar:
    member = next(m for m in tar.getmembers() if m.name.endswith("_ch0.sigmf-data"))
    raw = tar.extractfile(member).read()

L1 = np.frombuffer(raw, dtype=np.int8).astype(np.float32).view(np.complex64)
```

Replace `_ch0` with `_ch1` for the GPS L5 channel. The two channels have different sample rates (and hence different numbers of samples for the same duration), so always read each channel's rate from its own `.sigmf-meta` (`core:sample_rate`) rather than assuming a common one.

8 Checking signal levels and choosing `-gain`

TwinRX's RX gain is a fixed, manually-set Programmable Gain Amplifier (PGA) stage, not an AGC loop (UHD's generic `set_rx_agc()` call raises `NotImplementedError` on TwinRX) – so the right gain has to be chosen once and does not adjust itself during a recording.

Because GNSS signals sit 15–20 dB below the thermal noise floor, there is no "signal" to look for when picking a gain: the real goal is placing the composite *noise floor* correctly within the ADC's dynamic range – high enough that quantization noise does not matter, low enough that strong interferers do not clip the rails. `check_levels.py` reports exactly this, directly from an archive's raw samples:

```
uv run x310usrp_record.py --duration 10 --gain 20 25   # one gain per channel
uv run check_levels.py recordings/x310_<timestamp>.sigmf
```

It prints each channel's I/Q standard deviation against a target band (1/16 to 1/3 of full-scale: 128 for the current `int8` samples, i.e. 8 to 42; 32768 for older `int16` recordings) and the percentage of clipped samples, with a one-line verdict (raise/lower `-gain`, or OK). With a 20 dB external active LNA already in this lab's RF chain ahead of the USRP, the TwinRX gain is set to **25 dB** on both channels. Since the L5 channel's 20 MS/s bandwidth lets in about 7 dB more noise power than the L1 channel's 4 MS/s, the two channels sit at different levels for the same gain, so `check_levels.py` should be read per channel. This matters more with 8-bit samples, which have only ±127 of headroom: clipping shows up much sooner than with 16 bits; if one channel falls outside the target band, give it its own value via `-gain <ch0> <ch1>`. Re-run the check if the antenna, cable, or LNA changes.

9 Automating recordings with systemd

Scheduled, unattended recording is driven by a timer/service pair in `systemd/`, enforcing (independently of the timer's own schedule, so a manual trigger or a missed/caught-up run can never violate them):

- recording only inside the `[16:00, 18:00)` local-time window;
- at most two recordings per calendar day;
- the second recording starting at least 30 min after the first;
- sensing the USRP via `uhd_find_devices` before attempting to record, retrying silently every 15 min if it is not reachable;
- recording to a local staging directory (`staging/`, skipped if it has less than 35 GB free) and only then moving the finished archive to the NAS (`recordings/` links to it). Writing directly to the NAS over SMB stalls the receive loop for about 100 ms at a time, longer than the roughly 70 ms the L5 channel's receive buffer lasts at 40 MB/s, which drops samples. The archive is moved with `rsync -rt -remove-source-files`, which writes it under a hidden temporary name and renames it only once complete (so the NAS never shows a truncated `.sigmf`), and deletes the local copy only after a successful transfer; if the NAS is unreachable, the archive stays in `staging/` and is moved on the next run.

Manual recordings should likewise use a local directory, e.g. `uv run x310usrp_record.py -out-dir staging`, and be moved to the NAS afterwards.

`x310-record.timer` fires every 15 min between 16:00 and 18:00; `x310-record.sh` (invoked by `x310-record.service`) is what actually evaluates all of the conditions above and, if they pass, runs `x310usrp_record.py`. Install and enable it with:

```
sudo ln -s $PWD/systemd/x310-record.service /etc/systemd/system/
sudo ln -s $PWD/systemd/x310-record.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now x310-record.timer
```

Using symlinks rather than copying the unit files means editing them in the repository takes effect after a `daemon-reload`, with no separate install step. Check what is scheduled, and inspect a run's output, with:

```
systemctl list-timers x310-record.timer
journalctl -u x310-record.service
```

10 Fortnightly WhatsApp report

Every 15 days, a short text summary and a compiled PDF of this very report are sent over WhatsApp to the lab head, via a self-hosted OpenWA gateway. OpenWA automates the unofficial WhatsApp Web client (not Meta's official API), so its own documentation notes a real, non-zero account-ban risk; using a dedicated number rather than a personal one is recommended.

10.1 One-time OpenWA setup

```
git clone https://github.com/rmyndharis/OpenWA && cd OpenWA
docker compose -f docker-compose.dev.yml up -d
docker exec openwa-api cat /app/data/.api-key   # save as OPENWA_API_KEY
```

Create a session (the `"name"` given here is only a label; OpenWA assigns its own UUID, returned as `"id"`, which is what every later call – including `OPENWA_SESSION` below – must use):

```
curl -X POST http://localhost:2785/api/sessions \
     -H "X-API-Key: $OPENWA_API_KEY" -H "Content-Type: application/json" \
     -d '{"name": "x310-notifier"}'
```

Start it and fetch its QR code, then scan the QR with the WhatsApp account that should send the reports (WhatsApp app → Linked Devices → Link a Device):

```
curl -X POST http://localhost:2785/api/sessions/<uuid>/start -H "X-API-Key:
    $OPENWA_API_KEY"
curl http://localhost:2785/api/sessions/<uuid>/qr -H "X-API-Key: $OPENWA_API_KEY"
```

Finally, store the three secrets the report service needs in a `.env` file at the repository root (already excluded from version control):

```
OPENWA_API_KEY=<api key from above>
OPENWA_SESSION=<session uuid from above>
REPORT_WHATSAPP_TO=<recipient phone number, e.g. +55...>
```

10.2 `report_summary.py`

On each run, `report_summary.py` scans `recordings/` for every `.sigmf` archive newer than the last report (tracked in a state file, defaulting to 15 days back on first run); for those, it

1. sends a short WhatsApp text digest – recording counts, data volume, lost samples, and which days (if any) had no recording;
2. appends one subsection per day (holding one sub-subsection per recording that day, each a metadata table) and refreshes the recording-calendar table in `report/main.tex`;

3. recompiles `report/main.tex` and sends the resulting PDF as a WhatsApp document.

Run manually, loading the secrets from `.env`:

```
uv run --env-file .env report_summary.py
```

10.3 Automating the report

Analogous to the recording service, `x310-report.timer` fires `x310-report.service` on a *relative* schedule – exactly 15 days after the previous run, rather than approximating with calendar-day arithmetic:

```
sudo ln -s $PWD/systemd/x310-report.service /etc/systemd/system/
sudo ln -s $PWD/systemd/x310-report.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now x310-report.timer
```

11 Clock source, time source, and LO source

These three settings have different purposes. They are set directly as command-line flags to `x310usrp_record.py` (see also Section 9's underlying `uv run x310usrp_record.py` invocation).

11.1 Clock source

The clock source provides the precision frequency reference used by the radio clocking and synthesizers.

With an external 10 MHz reference connected to `REF IN`, use:

```
--clock-source external
```

The 10 MHz reference does not enter through an RF receive connector.

11.2 Time source

The time source establishes the timing reference for the radio time counter, typically through a 1 PPS signal.

Because no external PPS is connected in the present setup, use:

```
--time-source internal
```

If a 1 PPS signal were connected to the X310 PPS input, `-time-source external` could be used; `x310usrp_record.py` then also waits for a PPS edge to actually be detected (by checking that `get_time_last_pps()` advances between two reads) before recording, since UHD has no boolean "PPS locked" sensor the way it does for the reference clock and the LOs.

11.3 LO source

The LO is the RF local oscillator used by the mixer to translate the received RF signal to IF/baseband. `x310usrp_record.py` always uses each channel's internal LO synthesizer, referenced to the external 10 MHz signal selected above:

$$\text{External 10 MHz} \rightarrow \text{REF IN} \rightarrow \text{internal PLL/synthesizer} \rightarrow \text{RF LO} \rightarrow \text{mixer}$$

An external LO would instead mean supplying the actual RF LO signal from an external RF generator – not the same thing as the 10 MHz reference, which is the wrong frequency for the RF mixing stage.

12 Two-channel TwinRX tuning

TwinRX has two receive channels. Their LO configuration determines whether they can be tuned independently.

12.1 Same frequency with phase coherence

For two channels receiving the same RF frequency with phase coherence, LO sharing can be used: one channel generates the LO and the other uses it as a companion. This is appropriate when both channels observe the same RF band and relative phase is important. `x310usrp_record.py` does not use this mode.

12.2 Different frequencies

For independent center frequencies, both channels must use independent internal LOs – TwinRX phase synchronization must not be enabled. This is exactly the case here:

- Channel 0: GPS L1 = 1575.42 MHz;
- Channel 1: GPS L5 = 1176.45 MHz.

Because L1 and L5 are separated by about 399 MHz, they cannot be obtained from one shared analog LO using only a digital frequency offset; they require independent analog LO tuning, which is what each channel's own `set_rx_freq` call in `x310usrp_record.py` sets up.

13 Verifying lock status

Before streaming, `x310usrp_record.py` waits for the reference clock and both channels' LOs to report locked, raising a clear error after a timeout rather than silently recording against an unlocked radio. The same sensors can also be queried directly, independent of the Python script, with:

```
uhd_usrp_probe --args="addr=192.168.10.2" --tree | grep lo_locked
```

For this X310, the paths are (the exact path may vary with UHD version):

```
/blocks/0/Radio#0/dboard/rx_frontends/0/sensors/lo_locked
/blocks/0/Radio#0/dboard/rx_frontends/1/sensors/lo_locked
```

Check a given channel directly:

```
uhd_usrp_probe \
  --args="addr=192.168.10.2" \
  --sensor="/blocks/0/Radio#0/dboard/rx_frontends/0/sensors/lo_locked"
```

For a healthy configuration, this returns `true` for both channels – matching what `x310usrp_record.py` itself reports as `ch0 lo_locked:  locked` / `ch1 lo_locked:  locked` on its own console output during a normal run.

14 RFNoC block-factory warnings

When probing or using the radio, warnings such as the following may appear:

```
[WARNING] [RFNOC::BLOCK_FACTORY]
Could not find block with Noc-ID ...
```

These appear routinely during normal `x310usrp_record.py` runs on this FPGA image and have not prevented successful communication with the X310, streaming, or access to the TwinRX lock sensors. The radio still enumerates normally, and standard blocks such as the DDCs, DUCs, radios, replay block, and TwinRX daughterboard remain visible.

15 Recommended final configuration

For the current L1/L5 dual-frequency GNSS recording setup:

```
Host Ethernet IP:   192.168.10.1/24
X310 IP:            192.168.10.2
Ethernet:           1 GbE, port 0
MTU:                1500

Clock Source:       external
Time Source:        internal
LO Source RX0:      internal
LO Source RX1:      internal

TwinRX RX0:         GPS L1 = 1575.42 MHz
TwinRX RX1:         GPS L5 = 1176.45 MHz
Sample rate RX0:    4 MS/s  (L1)
Sample rate RX1:    20 MS/s (L5)
Gain RX0/RX1:       25 dB / 25 dB (Section~\ref{sec:gain})
Sample type:        sc8 / ci8 (int8 I + int8 Q = 2 bytes per sample)
Recording length:   300 s (5 min), up to twice per day, 16:00-18:00 local time
```

Equivalently, as the command this whole configuration reduces to when run manually:

```
uv run x310usrp_record.py
```

since every value above is already `x310usrp_record.py`'s own default. This configuration gives independent L1/L5 tuning while keeping both LO synthesizers referenced to the same precision external 10 MHz source, with unattended scheduling and reporting handled by the services in Sections 9 and 10.